

Shell programming .

TYPES OF SHELLS:

The shell is a program that runs the Unix system just like any other program. Several different flavors of shell have evolved.

Each of the shells slightly differ in features e.g. sh, csh, ksh, rcsh, etc.

The “Bourne Shell” (sh)

Inventor is Stephen Bourne. It is standard currently available with Unix system. The prompt is \$.

The “C Shell” (csh)

This shell resembles the C programming languages in syntax. The prompt is %. It is Berkeley shell.

The various commands/ parameters used by the shell are

1. echo
2. shell parameters
3. shell variables
4. read
5. expr
6. test

1. The echo command

Syntax

```
$ echo "HELLO"
```

The echo command is used to display messages on the screen. The echo command displays the enclosed text on screen. It is preferable to enclose the message to be enclosed within double – quotes. By default, the echo command displays the text and then puts a new line character at the end text. The newline after the text is displayed. Using certain characters in the command can change this default.

E.g.

```
$ echo "HELLO \007"
```

This example will display the text and then sound the system bell

Character	Functions
\c	Keeps the curser on the same line of screen after displaying the argument.
\n	Displays an additional blank line after arguments.
\007	Displays the arguments and then sound the systems bell on the next line.

2. Shell parameters

Special parameters used by the shell are

Shell parameters	Significances
\c	Keeps the curser on the same line of screen after displaying the argument.
\$0	The filename of the current script.
\$1, \$2,... etc.	The positional parameters
\$\$	The process number of the current shell. For shell scripts, this is the process ID under which they are executing.
\$*	Complete set of positional parameters as a single string
\$#	Number of arguments specified in command
\$?	Exit status of last command
#!	PID of last background job
\$@	Same as \$* except when enclosed in double quotes

Command-Line Arguments

The command-line arguments \$1, \$2, \$3, ...\$9 are positional parameters, with \$0 pointing to the actual command, program, shell script, or function and \$1, \$2, \$3, ...\$9 as the arguments to the command.

Following script uses various special variables related to the command line –

```
#!/bin/sh
echo "File Name: $0"
echo "First Parameter : $1"
echo "Second Parameter : $2"
echo "Quoted Values: $@"
echo "Quoted Values: $*"
echo "Total Number of Parameters : $#"
```

Here is a sample run for the above script –

```
$/test.sh Zara Ali
File Name : ./test.sh
First Parameter : Zara
Second Parameter : Ali
Quoted Values: Zara Ali
Quoted Values: Zara Ali
Total Number of Parameters : 2
```

3. Variables

The variables used by the shell are called shell variables. No type declaration or initializations are necessary before you can use them.

Creating variables:

Shell variables are assigned value with the = operator. They can be created at any point of time by a simple assignment of value. A variable can be created without a value by leaving right hand side of assignment operator blank

The syntaxes for creating a variable is

Variable=value

Note: There **must be no spaces on either side of the assignment operator(=)**, if the value is being assigned with the spaces, then it should be enclosed within quoted.

i.e. variable="HELLO SANIA"

if not, quotes are optional.

Variable=134

In above assignment, variable is not numeric variable, but character string. Therefore, the variable var contains the character 1 and not the 134.

Referencing variables

The \$ symbol is used to refer to the content of the variable.

Eg. to assign value of 1 variable to another

Syntax: variable1=\${variable2}

The braces are essentially used to delimit the variable name. Assume a variable today which contains the value 20. If you want another variable x to contain this value which will concatenate it with string 'th' i.e. (20th)

Then, x= \$todayth

Would not produce the desired result, because, shell would look variable called 'todayth '. Therefore, you would have to use braces as

X=\$ {today}th

Now x would content the value 20th..

4. The read command

It allows user to assign value to variable during execution of shell script. It makes scripts interactive. It is used with 1 or more variables, and input supplied through the standard input.

eg.

```
echo "Enter your name"
read my name
```

Note that, read command does not prompt the user to enter data. In order to do so, echo command must be used.

5. The expr Command

This command combines two functions in one. It can perform arithmetic operations on integers, and also manipulate strings to limited extent.

The expr can perform basic four arithmetic operations as ewll as modulus function:

```
$ x=3; y=5
$ expr 3 + 4
8
$ expr $x - $y
-2
$ expr 3 \* 5
15
$ expr $y \% $x
```

The operators supported by `expr` are `+`, `-`, `*`, `/` etc. must be enclosed on either side by white space. However, when multiplication/division is used, the symbols should be preceded by backslash. Otherwise the shell will interpret `*` as a wildcard character and `/` as a file separator. It is to be noted that `expr` does not support decimals.

6. The test command

It is used to **test values of variables**. When you utilize it to evaluate expressions, `test` statement is used as its control command. `Test` uses certain condition on its right, and returns either true or false exit status, which is then used by `if` for taking decisions.

Relational operators also have different forms, when used by `test`.

They always begin with a `-` (hyphen), followed by two character word, and are to be enclosed on either side with white space. The operators are significant mnemonics, `-eq` implies equal to. The complete set of operators is shown

Relational operators used by `if`

operator	Meaning
<code>-eq</code>	Equal to
<code>-ne</code>	Not equal to
<code>-gt</code>	Greater than
<code>-ge</code>	Greater than or equal to
<code>-lt</code>	Less than
<code>-le</code>	Less than or equal to

Ex.

```
$ x=5; y=7; z=7.2
```

```
$ test $x-eq $y; echo%?
```

```
1
```

Test: File tests

File related tests with `test`

operator	Exit status
----------	-------------

-e file	True if file exists
-f file	True if file exists and regular files
-r file	True if file exists and is read file
-w file	True if file exists and is write file
-x file	True if file exists and is executable file
-d file	True if file exists and is directory file
-s file	True if file exists and has size greater than 0.

The for.....do loop

One of most commonly used loop is for....do loop. It iterate through a list of values, executing the body of loop for each element in the list.

Syntax:

```

For variable in list
Do
    Loop body
Done

```

In the above example, for loop assigns the values in list to variable one at time. Then for each value, statement in body in braces between do and done are executed. Variable can be specified by any valid variable name. List can be composed of any values that can be defined because each item in list is treated as string value in the body of the loop.

e.g

```

$ for x is 1 3 4 6 7
> do
> echo "The value of x is $x"
> done
The value of x is 1
The value of x is 3
The value of x is 4
The value of x is 6
The value of x is 7

```

Each element in loop is separated from the next by while space

The if conditional loop :

Syntax:

```

If [condition]

```

```
Then
Command(s)
Else
Command(s)
Fi
```

In this type of loop, each of the expressions is tested. Based on the result of that test, one of the actions that follows is taken

e.g.

```
$ USER = "HELLO"
$ if [ $ USER = "HELLO" ]
then
echo $ USER,RAMAN
fi

HELLO,RAMAN
```

The while.....do

Some times we don't want to check for condition only once, we want to continue doing something while a condition is met, or is no longer true. You can do this with a "while" loop.

SYNTAX :

```
while condition
do
Command(s)
done
```

e.g.

```
$ COUNT = 1
$while [ "$COUNT" -le 10 ]
Do
Echo $COUNT
COUNT = 'expr $COUNT + 1'
```


Done

OUTPUT:

1 2 3 4 5 6 7 8 9 10

\$

The Case Statement

comparing a value against a whole series of values

SYNTAX:

Case value in

Pattern 1) command ;;

Pattern 2) command ;;

Pattern 3) command ;;

.....

esac (indicating of end case statement)

e.g if program file digit contains following

Examples:

Program to check whether the file name supplied from command line argument is a file or a directory

Step1. Write the following program in a file named test file.

```
$ vi testfile.sh
if test -d $1          # $1 is the first argument, checks if it is a directory
then
echo " This file is a directory !"
elif test -f $1        # $1 is the first argument, checks if it is a file
echo "This file is an ordinary file!"
fi
```

Step2. Type `chmod +x testfile` command to assign executable permission to the test file.

Step3. Create a file by typing `touch f1.txt` and create a directory by `mkdir d1`.

Step4. Run the above script by typing **\$sh test f1.txt** command.

Step5. Observe the output, and check if it shows correct result as:

OUTPUT :

This file is an ordinary file!